HeavyFinder: Efficient and Fine-grained Heavy Hitters Detection with Instantaneous Flow Rate

Xirui Li*, Yangyang Wang*, Ruihao Wang*, Jiahao Cao*, Qilong Shi*, Mingwei Xu*, Lihua Miao†
* Tsinghua University †Huawei Technologies Co Ltd

2025 IEEE/ACM International Symposium on Quality of Service (IWQoS) | 979-8-3315-4940-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/IWQOS65803.2025.11143517

***Abstract*—The detection of Heavy Hitters (HH) of flows in a network plays a crucial role in a variety of critical applications, including network topology optimization, congestion control, and network security (e.g., DDoS mitigation). In high-performance data centers, tasks such as optimizing model training and detecting anomalies require microsecond-level granularity for burst and congestion detection, demanding higher accuracy and fine granularity in HH detection. However, existing HH detection schemes primarily focus on traffic accumulation over longer periods and ignore instantaneous flow rates, making them incapable of detecting short-duration heavy hitters (at the granularity of milliseconds to microseconds) that can significantly affect network performance.**

This paper proposes a rate-sensitive definition of HHs and introduces HeavyFinder, a framework for enabling efficient HH detection of flows at microsecond granularity and accurately describing their traffic changes. It detects HHs based on inherent characteristics of both traffic volume and instantaneous flow rates, and improves the existing elephant flow filtering mechanism. Furthermore, this framework provides an efficient information aggregation method for reporting HH information, which can reduce overhead further. We deployed and tested HeavyFinder on x86 CPUs and evaluated its performance by using real network trace data. Results show that HeavyFinder achieves sub-10-microsecond accuracy in detection for the start and end times of HHs, with 3–10× lower reporting overhead compared to existing frameworks.

***Index Terms*—Heavy-hitters detection, Instantaneous flow rate, Microsecond granularity, Programmable switch**

I. Introduction

Heavy Hitters (HHs) refer to flow behaviors that consume significant bandwidth or resources, which have a substantial impact on network bandwidth, latency, and overall performance. Therefore, HH detection plays a critical role in network performance optimization [1], anomaly detection [2]–[4], and network security protection [5].

Since the introduction of new hardware [6] and the imbalanced development of device high-throughput and buffer capacity [7], microsecond-level flow fluctuation behaviors are widespread in modern data centers [8], [9], where bursts lasting only a few hundred microseconds can cause instantaneous network congestion [10]. Moreover, the latest DDoS attacks evade traditional coarse-grained monitoring frameworks by generating bursts with short duration and high rates [11]. Therefore, fine-grained HH detection is necessary to enable network administrators to more accurately identify flow behaviors that significantly impact network performance at a small time scale of microseconds, thereby addressing performance and security challenges in data centers more effectively. To achieve this, the HH detection framework must meet three key requirements:

(1) Accurate detection of HH: This involves the real-time identification of all HH events and the precise capture of their temporal boundaries and total traffic volume at microsecond-level granularity.

(2) Fine-grained description of HH: Since network anomalies can occur at any time, the framework must capture the precise rate fluctuation of HHs at any moment, rather than relying on simple average rates. It should be capable of describing flow behaviors at sub-millisecond granularity.

(3) Lower overhead: Since network measurement often requires full-network deployment, the detection framework must ensure that its total overhead remains below 0.02% of the network bandwidth [12] to ensure the detection overhead does not impair network functionality when deployed across multiple devices.

Unfortunately, existing HH detection techniques fail to meet these requirements, primarily due to limitations in how HHs are defined. Although HH definitions vary across studies, they generally fall into two categories: (1) one category defines HHs as flows exceeding a proportion of total traffic, but it requires access to the complete dataset, making it unsuitable for real-time applications and thus excluded from this paper; (2) the other category defines them as flows surpassing a threshold within a fixed time interval. Frameworks detecting this kind of HH definition typically employ time granularities on the order of tens of milliseconds, capturing only average rates and failing to detect microsecond-level flow behaviors. Although reducing the window size to a few microseconds can reflect instantaneous rates, this approach requires lots of messages of monitoring result reports to accurately describe its total traffic volume and duration for the same HH, resulting in a hundredfold increase in transmission overhead [8]. Recent research [3], [13]–[17] has mainly focused on heavy hitter detection algorithms and data structures at large time scales of seconds. While these works have significantly improved detection accuracy and reduced memory overhead, they still rely on traffic-volume-based definitions of HHs, making it hard to balance microsecond-level detection granularity and overhead (i.e., simultaneously meeting requirements 1, 2, and 3). Although Printqueue [18] and Umon [8] have substantially reduced bandwidth overhead for microsecond-level monitoring through compression algorithms, their overhead can still reach tens of Mbps. Consequently, in wide multi-machine deployments, the impact of this overhead on the network resources cannot be ignored, failing to satisfy requirement 3.

979-8-3315-4940-4/25/$31.00 ©2025 IEEE

Authorized licensed use limited to: Tsinghua University. Downloaded on September 28,2025 at 13:54:14 UTC from IEEE Xplore. Restrictions apply.

This paper argues the HH detection at the granularity of microseconds. It improves the definition of HH behavior at small time scale with consideration of instantaneous flow rate and proposes HeavyFinder, a cost-effective, fine-grained continuous monitoring framework for detecting HHs at microsecond-level granularity. Unlike traditional approaches that rely on manually defined or fixed-period monitoring time windows, our method uses intrinsic flow characteristics, such as packet size and inter-packet intervals, to identify potential HHs that appear at any time at the microsecond level, making it more sensitive to flow rates and better at detecting subtle flow behaviors. Additionally, by relying solely on flow characteristics, our approach ensures unique HH boundaries unaffected by rigid factors (e.g., predefined HH duration), avoiding the high overhead of repeated reporting in small windows. Our main contributions are as follows:

- We analyzed the goals of HH detection at fine-grained time granularity and propose a rate-sensitive HH detection framework tailored to the prevalent transient flow behaviors in modern data centers. By calculating the latest arrival time of the next packet based on packet size and inter-packet intervals, our framework determines HH boundaries without relying on manually defined time windows, enabling the detection of burst events at the microsecond level, an extremely fine granularity.
- We improved the existing elephant flow filtering method [19], enabling faster and more comprehensive acquisition of large flow information in the network. This enhancement reduced the framework's measurement error for HH time boundaries from tens of microseconds to just a few microseconds.
- We designed a compact storage mechanism based on rate variation for aggregating reporting HH information to further reduce the report overhead of HeavyFinder. By employing compression storage and reusing fields, the report overhead was reduced by at least 50% compared to direct data reporting.
- We implemented and evaluated HeavyFinder with other existing related HH monitoring methods on an X86 CPU and implemented a prototype on a tofino programmable switch with P4 language to verify its feasibility. The results show that HeavyFinder can achieve microsecond-level HH detection with a reporting overhead of only a few hundred kilobits per second.

II. Related Work

In this section, we review existing network measurements that can detect HH and analyze why they fail to meet the requirements mentioned in Section I. Fig.1 shows the difference between each method.

A. Packet-Based Monitoring Methods

Traditional frameworks based on sampled packets, such as SFlow [20] and NetSight [21], failed to meet requirement 1 at low sampling rates since they only capture a small fraction of the packets, leading to reduced accuracy. Conversely, increasing the sampling rate will result in prohibitively high overhead [2].

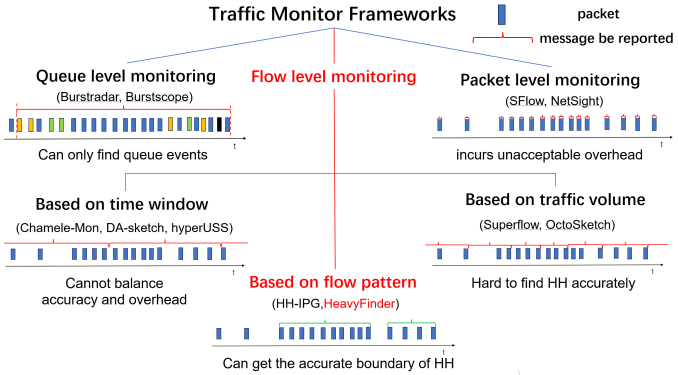


Fig. 1. **Comparison of existing methods**

B. Queue-Level Monitoring Methods

Schemes like Burstradar [22] and Burstscope [12] detect bursts based on queue information. It is important to note that the "bursts" they define based on queue thresholds are significantly different from the "bursts" of HH mentioned in this article. These bursts are detected at the queue granularity level and are independent of the specific flow rate. Consequently, they cannot reflect the behavior characteristics of a particular flow, failing to meet requirements 1 and 2.

C. Flow-Level Monitoring Methods

Most existing methods use sketches, a memory-efficient probabilistic data structure, to measure traffic over a given period of time. The methods represented by ChameleMon [3], DA-sketch [14], hyperUSS [15] improve the accuracy with innovative algorithms, while other methods represented by bitmatcher [17], Dhsketch [23] effectively reduce the overhead by introducing variable length fields. However, these methods perform poorly at extreme timescales [18], [24]. This is because they detect the boundary of flow behaviors based on an artificially specified time period rather than the characteristics of the flow itself. Thus, they can only achieve fine granularity in a very short reporting epoch, but this also introduces a great deal of overhead. So they cannot meet all three requirements at the same time.

Additionally, methods including Superflow [25] and OctoSketch [26] use counters to continuously report flows' key-value pairs to the analyzer based on traffic thresholds or filtering algorithms. These methods have lower reporting overhead than periodic reporting methods, since they avoid the repeated reporting of the whole data structure. However, they rely solely on traffic volume, making it difficult to accurately determine HH boundaries, failing to meet requirements 1 and 2.

Therefore, frameworks such as HH-IPG [24] and HeavyFinder combine multiple metrics, including rate and traffic volume, to better capture flow patterns. Although HH-IPG attempts to indirectly reflect flow rates through the exponentially weighted moving average (EWMA) of inter-packet gap (IPG), it cannot accurately determine the instantaneous rate of a flow only with IPG, and the

Authorized licensed use limited to: Tsinghua University. Downloaded on September 28,2025 at 13:54:14 UTC from IEEE Xplore. Restrictions apply.

EWMA itself introduces significant lag. Therefore, HH-IPG is insensitive to the instantaneous rates of flows and cannot capture short-term rate variations, so it can't meet requirements 1 and 2 either.

The most advanced frameworks, such as PrintQueue [18] and umon [8], can report the information of all the packets or tiny time windows (8us) under acceptable reporting overhead with innovative compression mechanisms. However, since they monitor the complete lifecycle of all flows, their report bandwidth remains relatively high, which can reach 1/1000 of the link capacity while deploying dozens of nodes. Additionally, these methods focus on different research priorities compared to our work, making us mutually orthogonal.

III. HeavyFiner

In this section, We'll briefly describe the key ideas of HeavyFinder and describe the workflow of it.

A. Key Ideas of HeavyFinder

1) Analysis of the Purpose of HH Detection: Heavy Hitter detection focuses on flows that significantly impact the network by identifying those consuming substantial resources. Compared to frameworks that monitor flows' traffic in their entire lifecycle, HH detection avoids the low-impact mouse flows, thereby it can balance accuracy and overhead. Thus, whether for anomaly detection or network optimization, HH detection aims to identify and accurately describe the characteristics of high-impact flows during specific time intervals.

However, traditional HH detection methods essentially determine HHs based on traffic volume over a fixed or periodical time window, which can't fully reflect a flow's influence on the network. As discussed in Section I, flows with low total traffic but extremely high rates within certain microsecond-level time frames can still significantly impact the network. For instance, LoRDMA [11] exploits this characteristic to trigger severe network congestion. Moreover, user-defined windows introduce strong subjectivity, which fails to capture the objective state of flows.

Therefore, we propose the definition of fine-grained HH detection with consideration of flow rate in data centers. It possesses the following three characteristics:

Using rate and volume as dual criteria: In addition to the traffic volume, the instantaneous rate of a flow is also a critical feature. It directly correlates with the flow's resource utilization (e.g., bandwidth-delay product). Thus, incorporating a rate threshold into the HH definition can accurately reflect the impact of fine-grained rate fluctuations on the network.

Ensuring flow continuity: Given the high bandwidth of data centers, packets accumulated in queues are quickly cleared [8]. Therefore, a HH must continuously send packets at a certain rate to maintain a high resource occupancy. Without this constraint, the framework might mistakenly identify two distant bursts as a single HH, thereby reducing detection accuracy.

Flow-driven unique boundaries: Manually defined time windows introduce subjectivity, making them unable to accurately represent the true start and end times of HHs in any time and resulting in non-unique measurement outcomes. Furthermore, they may lead to the misidentification or omission of HHs [24]. Therefore, the determination of HH boundaries should rely exclusively on the intrinsic characteristics of the flow itself.

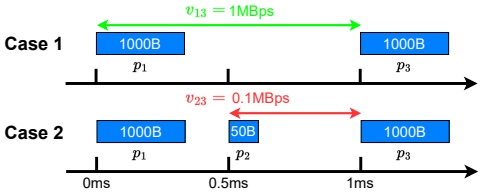


Fig. 2. **A paradoxical situation**

In summary, we can define the HHs we will monitor at a fine-grained granularity as flow behaviors whose rate consistently exceeds a predefined threshold (vth) and whose cumulative traffic surpasses a minimum value ($minSize$). However, due to the absence of manually specified time windows, this definition introduces several practical challenges in monitoring, including how to calculate rates, determine the continuity of HHs, and accurately identify HH boundaries.

2) Definition and Detection Methods of Heavy Hitters: To address the aforementioned challenges, we first consider the information that can be obtained in real-time. Suppose there is a flow f consisting of m packets $f = \{p_1, p_2, ..., p_m\}$. When packet p_i arrives, the available information includes its arrival time $p_i.ts$, package size $p_i.size$. Without using pre-defined time windows, we calculate the average rate between any two packets by dividing the traffic volume by the time interval. To avoid overestimating the rate, we exclude the last packet's size from the calculation, thus the rate between p_i and p_j is $v_{ij} = \frac{\sum_{k=i}^{j-1} p_k.size}{p_i.ts - p_j.ts}$ $(i < j)$. When it comes to the continuity of HH, we refer to existing sliding-window-based monitoring method [13] and define HH as a sequence of packets where the average rate between any two packets exceeds a threshold vth. However, this definition is too strict and can result in a paradoxical situation, as shown in Fig. 2, assuming that $vth = 1MBps$ and the addition of small packet p_2 in case 2 makes $v_{23} < vth$, the packet sequence is no longer continuous, even if the total traffic in case 2 is greater than case 1. Therefore, we relax the restriction of HH and only require the existence of a subsequence containing the first and last packets, where the average rate between any two packets is greater than the threshold. Finally, to ensure that HH has a unique boundary, we search for the longest subsequence that satisfies Features 1 and 2, and define the HH's start and end times as the arrival times of the first and last packets of this subsequence.

In practical detection, since it is not feasible to calculate the rate between every pair of packets, we transform this problem into a timeout detection problem. Specifically, when each packet arrives, we check whether it is timeout. If it does, it indicates that the instantaneous rate of the flow is below the threshold. The simplest approach is to set a fixed timeout interval max_deltat, which has been used in existing works [27]. However, this only works with fixed

Authorized licensed use limited to: Tsinghua University. Downloaded on September 28,2025 at 13:54:14 UTC from IEEE Xplore. Restrictions apply.

packet sizes. To address this, we set the timeout interval to $packet_size/vth$. Nevertheless, this approach only captures the rate between adjacent packets, it can't resolve the special situation in Fig. 2. Therefore, we update and store the maximum timeout time $maxt$, ensuring that there exists a subsequence within HH where the rate between any two adjacent packets exceeds vth. We assume there is a subsequence of HH $\{p_i, p_{i+1}, ..., p_j\}$, as long as the average rate between any two adjacent packets exceeds the vth, the rate between any two packets will also exceed it. This is because for any two packets $k1$, $k2$ in the subsequence ($i < k1 < k2 < j$), there are $p_{k1}.size \geqslant vth\,(p_{k1+1}.ts - p_{k1}.ts)$, $p_{k1+1}.size \geqslant vth\,(p_{k1+2}.ts - p_{k1+1}.ts)$ ..., so there is:

$$p_{k1}.size + p_{k1+1}.size + \cdots + p_{k2-1}.size \geqslant vth\,(p_{k2}.ts - p_{k2-1}.ts + p_{k2-1}.ts - \cdots - p_{k1}.ts)$$

thus we have:

$$\frac{\sum_{k=k1}^{k2-1} p_k.size}{(p_{k2}.ts - p_{k1}.ts)} \geqslant vth$$

Besides, any two packets contain the situation of adjacent packets, thus proving that the monitoring method is equivalent to the definition method.

In summary, when packet p arrives, we assess whether it has timed out by comparing its timestamp ($p.ts$) with the latest arrival time ($maxt$). If it isn't timed out, we dynamically update $maxt$ for the next packet using the specified formula $maxt = \max(maxt, p.ts + p.size/vth)$ and revise the recorded size and end time of the HH with $p.size$ and $p.ts$. If a timeout is detected, we determine whether the HH's size exceeds the predefined minimum threshold. If it does, the flow is identified as having generated a HH.

In addition, to ensure accurate characterization of HHs, a long-duration HH will be segmented into multiple parts based on a fixed traffic threshold ($changeSize$). Each segment is represented by a flowlet called HH message (f, $size$, st, ed), where f is the 5-tuple of the flow and $size$ is the total traffic in this part of the HH. st and ed, respectively, express the start time and end time of this part of HH. A single HH may correspond to multiple pieces of HH message.

B. Workflow of HeavyFinder

The workflow of HeavyFinder consists of two components: the detection part for HH detection and the analysis part for network-wide analysis. The detection part can be deployed either on the host or within the data plane of programmable switches, while the analysis part should be deployed on a centralized analyzer for comprehensive network analysis. All subsequent steps will be elaborated in detail in Section IV.

(1) Filter the elephant flows: Due to resource limitations, we can only track a subset of flows. When a flow arrives, we first determine whether it is an elephant flow. If it is, we consider it a potential HH producer and track it; otherwise, we treat it as a mouse flow and perform no further actions.

(2) Detect and Describe HH: For tracked elephant flows, we detect whether they produce HH according to the method described in Section III-A2, and generate HH messages in the form of flowlets.

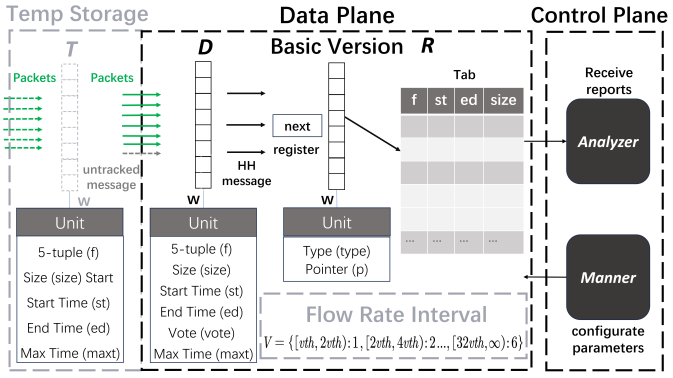


Fig. 3. **HeavyFinder architecture**

(3) Compress and Report: The collected HH messages will be compressed for efficient storage and reporting. HH messages will be reported to the central analyzer once it reaches a predefined threshold.

(4) Analyze the HH information: The central analyzer stores the reported HH messages and uses them to address specific issues such as congestion control, DDOS attack detection, and SLO violation attribution upon user or application requests.

IV. HeavyFinder Design

In this section, we provide a detailed description of HeavyFinder's approach to HH detection. We first introduce the basic implementation of HeavyFinder in Section IV-A, followed by two significant optimizations in Section IV-B Finally, we present its hardware implementation in Section IV-C For parameter configuration, we assume that the rate threshold is vth, the minimum traffic threshold is $minSize$, and that a HH message is generated when the traffic volume exceeds $changeSize$.

A. Design of Basic HeavyFinder

In this section, we present the basic data structure and algorithm of HeavyFinder, which has two primary operations: update and storage.

1) Basic Data Structure of HeavyFinder: As shown in Fig.3, HeavyFinder is composed of two modules: the detection module (D) and the reporting module (R). The detection module is responsible for tracking and identifying potential HH, which consists of w buckets and each bucket contains six fields. Among them, f, $size$, st, ed are used to record the flow identifier (five-tuple), total traffic volume, start time and end time of a flow segment whose rate consistently exceeds vth. The field $vote$ is used to implement majority vote [19] to filter out elephant flows, while $maxt$ is utilized to determine whether the flow's rate remains above vth continuously.

The reporting module (R) is mainly responsible for recording the HH messages generated by the detection module and reporting them to the analyzer effectively. It consists of a flowlet list Tab of length n, a pointer allocation register $next$ to assign positions for HH messages within the list, and w buckets, each containing only a field $type$, to indicate the type

Authorized licensed use limited to: Tsinghua University. Downloaded on September 28,2025 at 13:54:14 UTC from IEEE Xplore. Restrictions apply.

of the previous stored HH message. For clarity, we denote field y of the ith bucket in module x as $x.y$.

Algorithm 1 Update

Input: packet p
Output: $change_flag$,$HH_message$
1: $i \leftarrow hash(p.f)$, $change_flag \leftarrow 0$
2: $maxt \leftarrow p.ts + p.size/vth$
3: **if** $D[i].f == p.f$ **then**
4: **if** $p.ts \leqslant D[i].maxt$ and $D[i].size \leqslant changeSize$ **then**
5: $D[i].size \leftarrow D[i].size + p.size$
6: $D[i].ed \leftarrow p.ts$
7: **else if** $p.ts \geqslant D[i].maxt$ **then**
8: $change_flag \leftarrow 2$
9: **else if** $D[i].size \geqslant changeSize$ **then**
10: $change_flag \leftarrow 3$
11: **end if**
12: $D[i].maxt \leftarrow max(D[i].maxt, maxt)$
13: **else**
14: update $D[i].vote$ by $p.size$ and test whether D[i].f needs to be replaced by majority vote
15: **if** need replace **then**
16: $change_flag \leftarrow 1$
17: **end if**‘
18: **end if**
19: **if** $change_flag == 1$ **then**
20: $HH_message \leftarrow D[i].\{f, size, st, ed\}$
21: $D[i] \leftarrow (p.f, p.size, p.ts, p.ts, maxt)$
22: **else if** $change_flag == 2$ or $change_flag == 3$ **then**
23: $HH_message \leftarrow D[i].\{f, size, st, ed\}$
24: $(D[i].size, D[i].st, D[i].ed) \leftarrow (p.size, p.ts, p.ts)$
25: **end if**
26: **return**

2) Update Operation of HeavyFinder: Algorithm 1 illustrates the update operation of HeavyFinder. Initially, all fields are set to 0. When a packet p arrives, we first compute its index i in the detection module based on $p.f$ and calculate the latest arrival time $maxt$ for the next packet (lines 1–2). Subsequently, we check whether p conflicts with the tracked flow (line 3). If no conflict occurs and flow f does not meet the conditions for generating HH message, we perform a normal update on $D[i].size$ and $D[i].ed$ (lines 4–6). HH message is generated under two conditions: first, if $p.ts$ exceeds $maxt$ (line 7), it indicates that the flow rate has dropped below vth, marking the end of a potential HH; second, if $D[i].size$ exceeds $changeSize$ (line 9), as described in Section III-A2, to ensure the granularity in HH description, we segment the HH when $D[i].size$ surpasses $changeSize$. In both cases, we generate a HH message (f, $size$, st, ed) based on the information in bucket $D[i]$ (line 22–23) and send it to module R. After that, we will directly assign the information of p to the corresponding fields in $D[i]$.

If the packet p conflicts with the tracking flow (i.e., $p.f! = D[i].f$), the vote value is adjusted according to the majority vote [19], and a decision is made on whether to replace the currently tracking flow $D[i].f$. If a replacement is considered necessary, it indicates that a larger flow has been mapped to this bucket, triggering the generation of a heavy hitter (HH) message and the updating of relevant fields in the register (lines 19–21) with the values from p. The key distinction between this scenario and cases where $change_flag = 2$ *or* 3 lies in the fact that it also replaces the flow $D[i].f$ recorded in $D[i]$.

Algorithm 2 Storage

Input: $change_flag$,$HH_message$,i
Output: $NULL$
1: $old_type \leftarrow R[i].type$
2: $R[i].type \leftarrow change_flag$
3: **if** $HH_message.size >= p.f$ or $old_type == 3$ **then**
4: $next+ = 1$
5: $idx \leftarrow next$
6: record $HH_message$ in $Tab[idx]$
7: **end if**
8: **return**

3) Storage Operation of HeavyFinder: After receiving a HH message, we first retrieve the state of the previous HH (old_type) in $R[i].type$ and rewrite it by using $change_flag$. If the total traffic volume of the HH message exceeds $minSize$, it will be recorded in Tab. It is important to note that when old_type is 3, even if the size of the HH message is smaller than $minSize$, it will still be recorded. This is because a value of 3 indicates that the previous HH message was generated due to the traffic exceeding $changeSize$. Since the total traffic volume of a HH is random, the size of the last HH message in a HH may likely be smaller than $minSize$, yet it still constitutes a part of the HH.

B. Optimizations

In this section, we will discuss two significant optimizations for HeavyFinder.

1) Optimized Elephant Flow Filtering Mechanism: Due to the memory constraints in network measurement scenarios, we can only track a finite number of flows simultaneously. Therefore, it is essential to implement an efficient elephant flow filtering algorithm to promptly determine which flows to monitor. As a classic algorithm widely adopted in various frameworks, the majority vote [19] uses the field $vote$ to record the traffic of untracked flows assigned to the same bucket. It then compares this traffic with the traffic of the tracking flow to identify the biggest flow (elephant flow). However, this approach relies on traffic volume for elephant flow filtering, which requires untracked flows to accumulate traffic over an extended period before being tracked. During this time, new packets are not recorded, causing flow information loss. In extreme cases, certain bursts may not be recorded at all.

To address these issues, we designed a rate-based voting mechanism combined with a temporary storage mechanism to rapidly and comprehensively filter high-speed flows. Unlike majority voting, the rate-based mechanism adjusts its weight based on the flow's instantaneous rate, speeding up the filtering of high-speed flows. However, this requires obtaining the instantaneous rate of untracked flows, which seems contradictory since the packet intervals of untracked flows are unavailable. To resolve this, we introduced a temporary storage mechanism, which fully retains the information of an untracked flow until the arrival of another flow's packet, thereby providing the

Authorized licensed use limited to: Tsinghua University. Downloaded on September 28,2025 at 13:54:14 UTC from IEEE Xplore. Restrictions apply.

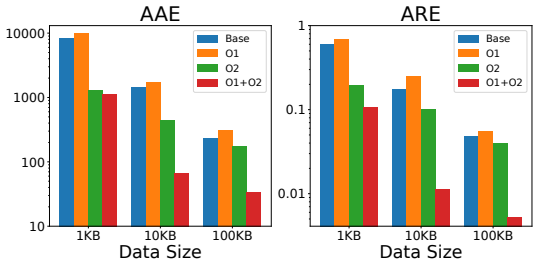


Fig. 4. **AAE and ARE of HH detection under different optimization. EL: basic EL framework. O1: Only use the temporary storage mechanism. O2: Only use the rate-based voting mechanism with basic information. O1+O2: combine the two mechanisms together**

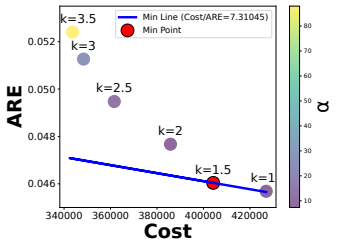


Fig. 5. **ARE_vs_Cost**

interval between packets for the rate-based filtering. Although temporary storage consumes additional memory and can only record the untracked flows for a short duration, this drawback is mitigated by the accelerated filtering enabled by the rate-based voting mechanism. Consequently, these two optimizations complement each other effectively.

As for the implementation, we introduce a temporary storage table T to implement the temporary storage mechanism. Similar to the bucket in D, T consists of the fields f, $size$, st, ed and $maxt$, where $maxt$ is used to ensure that the rates of flows stored in T are consistently higher than vth. Similar to the operations performed by the detection module, when a packet arrives, we first check whether the replacement conditions are satisfied (timeout or $p.f! = T[i].f$). If not, we calculate the inter-packet gap Δt by $p.ts - T[i].ed$, update $T[i].size$ and $T.ed$ and then change the vote value by τ, which can be calculated by $\frac{p.size^2}{\Delta t {v_{th}}^2}$. If $p.f! = D[i].f$ we plus τ to $vote$. Otherwise, subtract τ from $vote$. From Equation 1, it is evident that τ is proportional to the square of the flow's instantaneous rate.

$$\frac{p.size^2}{n\Delta t {v_{th}}^2} = \frac{1}{n} \times \frac{p.size^2}{\Delta t^2} \times \frac{\Delta t}{{v_{th}}^2} = \frac{\Delta t}{n} \times \frac{v^2}{{v_{th}}^2} \quad (1)$$

If the replacement conditions are satisfied, the temporarily stored flow information is passed to D as an untracked message in a manner similar to HH messages. Then we will determine whether to replace tracking flow, If the vote exceeds a predefined threshold θ, the tracked flow will be replaced and we update D[i] by untracked message.

Fig. 4 shows the accuracy of HH detection under different optimizations. When only the filtering mechanism is used (O1), HeavyFinder's detection accuracy is slightly lower than the baseline (Base) due to the additional space occupied by the filtering table. When only the majority vote is improved (O2), it significantly optimizes HH detection performance under limited memory but shows less improvement under ample memory. However, when both mechanisms are combined (O1+O2), they drastically reduce the detection error across all memory conditions, demonstrating their complementary nature.

2) Rate-Based Compression for Reporting Information: Although multi-segment reporting in Section III-A2 provides fine-grained HH information, it may result in a significant amount of redundant information. Since the main purpose of multi-segment reporting is to capture flow behavior changes [8], if a flow's rate remains constant, dividing it into segments is ineffective. Therefore, we propose a rate-based aggregation storage mechanism that groups HH messages with similar rates, thereby describing the rate distribution of HH with minimal overhead. This is achieved by dividing the rate into multiple intervals and aggregating HH messages falling within the same rate interval. It is notable that this aggregation is only for HH messages belonging to the same HH.

The rate interval V is represented as Equation 2, consisting of the intervals and their ID. Since flow rates can vary greatly in different scenarios, we focus on the proportion of rate change rather than its absolute value when dividing rate intervals. Thus, we use geometric progression to divide the rate intervals. For example, assuming the ratio is K, the length of the interval with ID 2 is K times the length of the interval with ID 1, and so on. We evaluate the performance of different divisions by calculating the scaling factor α, which is the normalized cost divided by the normalized reduction in ARE. A lower α indicates a larger ARE reduction with less cost. Fig 5 shows the distribution of ARE and reporting overhead $cost$ under different K values for HH detection. It can be seen that when $K = 1.5$, the framework has the smallest α, so we set $K = 1.5$ as the default.

$$V = \{[v1, v2) : 1, [v2, v3) : 2, [v3, v4) : 3, ..., \} \quad (2)$$

In terms of implementation, no new fields are needed in R. We simply modify the definition of $type$ to store the rate interval information of the previous HH message. In addition to the original values 1 and 2, we map numbers 3, 4, 5, 6 to intervals 1, 2, 3, 4 and so on. For Algorithm 2, its operations remain unchanged when old_type equals 1 or 2. However, when $old_type \geqslant 3$, the algorithm will check whether the previous and current HH messages fall within the same rate interval ($old_type == R[i].type$). If so, the messages are aggregated; otherwise, a normal operation is performed. During the aggregation process, we first retrieve the index idx from $R[i].pointer$, then update $Tab[idx].ed$ using $HH_message.ts$, and accumulate $HH_message.size$ into $Tab[idx].size$. This compression method can reduce the reporting overhead by 30%-91%, and with the additional compression of report fields and reuse of quintuple fields, it can further decrease reporting overhead by 50% to 95%.

C. Hardware implementation

This section focuses on the limitations of programmable switches [28] and describes how to implement the HeavyFinder introduced in Section IV on them.

Authorized licensed use limited to: Tsinghua University. Downloaded on September 28,2025 at 13:54:14 UTC from IEEE Xplore. Restrictions apply.

1) Limitation in Operate: Programmable switches can only support simple operations, and the number of computations in each stage is also limited. Thus, it is difficult for hardware to directly calculate complex formulas such as $\frac{p.size^2}{\Delta t v_{th}{}^2}$. To resolve this problem, we adopt the optimal piecewise constant approximation and use match-action tables to implement the corresponding algorithms. The keys and values in the table have been precomputed on the control plane.

2) Single-level Memory Access: In programmable switches, a packet can only access each field once in a given order. However, most operations of HeavyFinder can be completed in a single read-write cycle through the rational design of register sequences. For interdependent fields (e.g., $vote$ and f), we use the method in Elastic sketch [29] to implement them in the high and low bits of a register for simultaneous read and write.

3) Resource Usage of Hardware: We have deployed HeavyFinder on a P4 programmable switch [28], and the resource overhead is shown in Table I. However, due to the outdated version of hardware, certain operations supported by newer versions might lead to undefined behavior; thus, we defer further experiments to future work.

TABLE I
RESOURCE USAGE AND PERCENTAGES

Resource	Usage	Percentage
Exact Match Input xbar	136	10.62%
Hash Bit	518	12.45%
Gateway	30	18.75%
SRAM	33	4.12%
Map RAM	26	5.42%
TCAM	14	5.83%
VLIW Instr	26	8.12%
Stateful ALU	13	32.50%

V. EXPERIMENTS

A. Experiment Settings

1) Testbed: We implemented HeavyFinder on a computer equipped with an Intel i9-13900 CPU at 5.4 GHz and 16 GB of DRAM and completed the relevant experiments with this method under the Ubuntu 20.04 system. We also implemented a prototype of HeavyFinder on the Tofino switch.

2) Datasets: We evaluate HeavyFinder by using the following three traces from different scenarios:

MAWI Dataset: The MAWI-2023 [30] traffic trace spans approximately 15 minutes and comprises 250 million packets, with an average throughput of 2.7 Gbps. This dataset represents a typical network application scenario.

IMC Dataset: The IMC [31] dataset spans a duration of 20 minutes with approximately 15K packets per second, so its average flow rate is lower than real-world application scenarios. To address this, we accelerated the dataset by a factor of 50, increasing its throughput to approximately 5 Gbps. The acceleration also introduced numerous high-rate but short-duration bursts. Consequently, this dataset represents scenarios with frequent microbursts.

RDMA Dataset: To evaluate the frameworks' capability in monitoring RDMA flows, we generated RDMA traffic using the HPCC [32] code in an NS3 simulation. The simulation was configured with a 100 Gbps throughput with an 8-node topology. The link load induced by the workload was set to 30%, and the generated traffic conforms to the WebSearch flow size distribution.

3) Algorithms and Configurations: We implemented HeavyFinder in Python and compared it with several periodic frameworks, including Elastic Sketch (EL), Hashflow (HA), USS (USS), and Dasketch (Da), as well as continuous reporting frameworks such as Superflow (SF), HH-IPG (IPG), and Octosketch (OC). EL, USS, Da, HA, SF, and OC were implemented in Python based on the descriptions in the respective papers and C++ code, while HH-IPG was directly used from its original source code. Except for the window size and reporting traffic threshold, all other configuration of the frameworks were set to default values as recommended in corresponding papers. The parameters used by HeavyFinder and their default values are shown in Table II, and we will discuss the rationale behind these choices later. For all periodic algorithms, the window size was set to 1 ms, which is smaller than the typical 50 ms used in existing literature for such schemes. The warm-up time for HH-IPG was set to one-fifth of the window size (200 μs) to ensure accuracy. The reporting traffic threshold for Octosketch was set to 3,000 by default, which is the same as HeavyFinder's $minSize$. Superflow did not require any additional parameter configuration due to its design.

TABLE II
PARAMETERS OF HEAVYFINDER

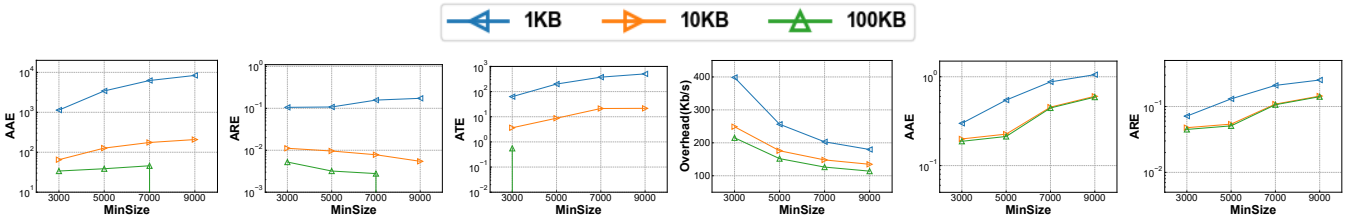
Name	*value in MAWI and IMC*	*value in RDMA*
θ	*100*	*100*
minSize	*3000byte*	*150000byte*
changeSize	*10000byte*	*500000byte*
vth	*1MB/s*	*50MB/s*

4) Tasks and Metrics: Two tasks are used to evaluate the algorithms:

Heavy Hitter Detection: To evaluate the accuracy of existing frameworks in detecting HH boundaries and traffic volume, we compare each algorithm's results to a benchmark. The benchmark is the ideal HH detection method, as defined in Section III-A2, which has unlimited memory and complete packet sequence information. Due to the significant differences in throughput across datasets, we set the rate threshold vth to 1 MB/s for MAWI and IMC datasets and 50 MB/s for the RDMA dataset.

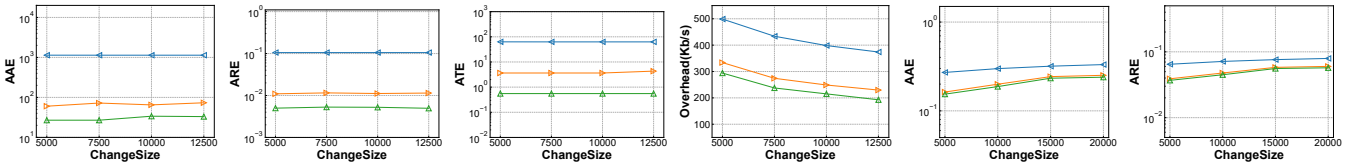
Heavy Hitter Description: To evaluate the algorithms' accuracy in characterizing high-speed flows, we randomly select fixed-length time intervals (t_range) for each flow, during which the total traffic volume exceeds $min(1/3\ minSize, vth \times t_range)$ (i.e., high rate and volume at the same time). We then evaluate the algorithm's accuracy in estimating traffic within these intervals. The rate threshold (vth) remains consistent with that used in Task 1. To account for variations in average flow rates, t_range is set to 5 ms for the MAWI and IMC datasets and 10 μs for the RDMA dataset. Although users can identify large flows directly from the flowlets reported by HeavyFinder, this task allows for

Authorized licensed use limited to: Tsinghua University. Downloaded on September 28,2025 at 13:54:14 UTC from IEEE Xplore. Restrictions apply.



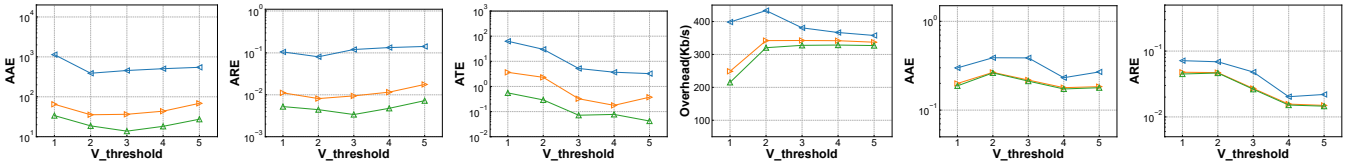
(a) AAE (Task1) (b) ARE (Task1) (c) ATE (Task1) (d) Overhead (Task1) (e) AAE (Task2) (f) ARE (Task2)

Fig. 6. **Accuracy and overhead vs. minSize**



(a) AAE (Task1) (b) ARE (Task1) (c) ATE (Task1) (d) Overhead (Task1) (e) AAE (Task2) (f) ARE (Task2)

Fig. 7. **Accuracy and overhead vs. changeSize**



(a) AAE (Task1) (b) ARE (Task1) (c) ATE (Task1) (d) Overhead (Task1) (e) AAE (Task2) (f) ARE (Task2)

Fig. 8. **Accuracy and overhead vs. vth**

a fairer comparison with other algorithms and highlights HeavyFinder's refined HH definition.

We use the following metrics to evaluate the performance of the algorithms in these tasks:

AAE: which is defined as $\frac{1}{|n|}\sum_{i=1}^{n}\sum_{j=1}^{segi}|f_{ij}-\widehat{f_{ij}}|$ where f_{ij} is the real traffic in the jth HH (or time period) of ith flow. As its estimate, $\widehat{f_{ij}}$ is calculated by weighting the size of the reported message, where the weight is the proportion of the overlap between the reporting time period and the actual time period relative to the reporting period. The unit of AAE is B.

ARE: ARE is defined as $\frac{1}{|n|}\sum_{i=1}^{n}\sum_{j=1}^{segi}\frac{|f_{ij}-\widehat{f_{ij}}|}{f_{ij}}$, and it can better reflect the average measurement accuracy of frameworks.

ATE: ATE represents the average time error, which is defined as $\frac{1}{|n|}\sum_{i=1}^{n}\sum_{j=1}^{segi}|\frac{|t_{ij}^{start}-\widehat{t_{nearest}^{start}}|+|t_{ij}^{end}-\widehat{t_{nearest}^{end}}|}{2}$. Its numerator is the sum of the absolute time differences (in microseconds) between the real HH's start or end times and the closest HH message's start or end times.

overhead: The overhead mentioned here refers only to the reporting cost with units in Kb/s.

B. Param Turning

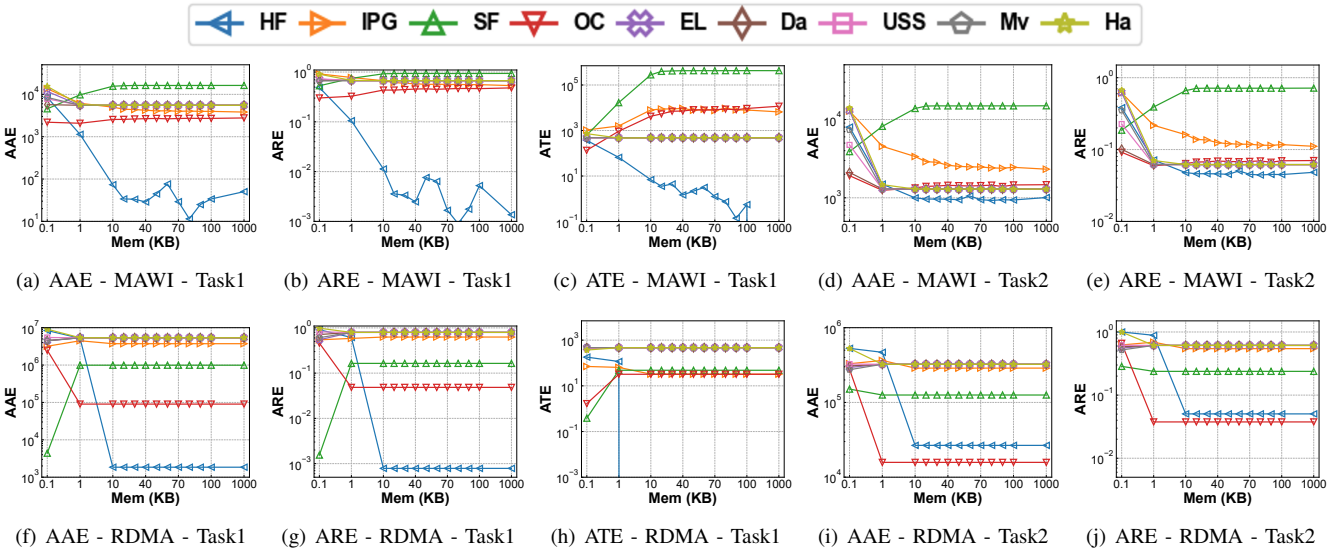
In this part, We changed the parameters of HeavyFinder on MAWI and observe the effect of the parameters on it.

1) MinSize: As the minimum traffic threshold of HH, $minSize$ significantly impacts the detection and description of HH. A smaller $minSize$ allows HeavyFinder to identify more subtle bursts but also increases reporting overhead, while a larger $minSize$ reduces overhead at the cost of higher errors. Fig. 6 shows our experiments with $minSize$ ranging from 3000B to 9000B, measuring AAE, ARE, and ATE across different tasks. The results show that increasing $minSize$ leads to higher HH description errors and lower overhead, which is consistent with our analysis. For HH detection, Figs. 6(a) and 6(b) show that detection errors increase as $minSize$ increases under low memory conditions but decrease under high memory conditions. This is because, with limited memory, hash collisions hinder flow tracking, while ample memory allows elephant flows to be stably tracked, making it easier for their traffic achieve $minSize$. For parameter configuration, $minSize$ should be a small fraction of the switch's buffer size.

2) ChangeSize: As the traffic threshold for segmented reporting, $changeSize$ does not participate in HH boundary determination. Therefore, it primarily affects reporting overhead and HH description granularity, with minimal impact on HH detection. In Fig. 7, we adjusted $changeSize$ from 5000B to 12500B and measured HeavyFinder's performance across different tasks, with results aligning with expectations. Since $changeSize$ primarily controls description granularity, it is recommended to configure it based on the user's desired measurement granularity and ensure it is not smaller than minSize.

3) Vth: As a critical component of HH definition, flow rate is closely related to detection accuracy and overhead. A higher vth can eliminate small flows and improve the detection accuracy of elephant flows. However, excessively high vth may relatively downgrade elephant flows to mouse flows, thereby reducing their accuracy. Besides, although high-rate flows can reduce the number of HH messages before aggregation, changes in the rate interval can also prevent aggregable HHs from being combined, which increases overhead. In Fig

Authorized licensed use limited to: Tsinghua University. Downloaded on September 28,2025 at 13:54:14 UTC from IEEE Xplore. Restrictions apply.



(a) AAE - MAWI - Task1 (b) ARE - MAWI - Task1 (c) ATE - MAWI - Task1 (d) AAE - MAWI - Task2 (e) ARE - MAWI - Task2

(f) AAE - RDMA - Task1 (g) ARE - RDMA - Task1 (h) ATE - RDMA - Task1 (i) AAE - RDMA - Task2 (j) ARE - RDMA - Task2

Fig. 9. **Detection and description accuracy estimation**

8, by adjusting the rate threshold vth from 1 MB/s to 5 MB/s, the results show that vth has complex effects on both detection accuracy and overhead. In practice, it is recommended to set vth to a small fraction of the device's throughput.

C. Heavy Hitter Detection

Fig. 9(a), 9(b), 9(c), 9(f), 9(g), 9(h) compares the accuracy of HF and other algorithms in heavy hitter detection on the MAWI and RDMA datasets. Since the results for the IMC dataset are similar to those of the MAWI dataset, we only show the results of MAWI. The memory range is set from 0.1 KB to 100 KB. Results show that, on both datasets, HF's AAE, ARE, and ATE outperform all other frameworks when the memory is greater than 1KB, achieving an advantage of 1-2 orders of magnitude over existing algorithms. However, when the memory is less than or equal to 1 KB, HF's accuracy decreases due to its larger bucket size and caching mechanism compared to other algorithms, which leads to significant hash collisions in scenarios with extremely limited memory. However, when memory is sufficient, HeavyFinder's ARE consistently exceeds 99%, with time errors below 10 microseconds.

D. Heavy Hitter Description

Figures 9(d), 9(e), 9(i), 9(j) compare the accuracy of HeavyFinder and other algorithms in estimating the traffic of high-speed flows. The memory range is also set from 0.1 KB to 100 KB. On the MAWI dataset, similar to Task 1, HF exhibits higher accuracy than other frameworks when memory exceeds 1 KB. However, the advantage is not significant enough to make a substantial difference. On the RDMA dataset, HF outperforms periodic-based frameworks, IPG, and SF by a significant margin, although its accuracy is slightly lower than OC. This is because both HF and OC segment flows for reporting based on flow traffic, resulting in finer monitoring granularity as the flow rate increases. However, without a compression mechanism, OC generates unacceptable reporting overhead, which we will demonstrate in Section V-E.

TABLE III
OVERHEAD OF OTHER METHODS

Framework	Granularity	Data Sets		
		MAWI	IMC	RDMA
Elastic Sketch	1ms	8500	6375	6375
	10ms	1912	1700	637
	50ms	680	765	null
Da Sketch	1ms	8500	6375	8500
	10ms	850	1275	850
	50ms	297	382	null
USS Sketch	1ms	10625	6375	6375
	10ms	5312	2337	637
	50ms	3782	1360	null
MV Sketch	1ms	10625	8500	14875
	10ms	1487	1912	1487
	50ms	425	510	null
HashFlow	1ms	12750	10625	12750
	10ms	850	1275	850
	50ms	297	382	null
SuperFlow	null	56	193	3280
HH-IPG	1ms	953	1039	2952
Octo Sketch	3000B	1422	2443	115948
HeavyFinder	3000B	226	449	5112

E. Overhead Analyzing

We implemented periodic reporting frameworks (EL, Da, USS, MV, and HF) with different window sizes. Since these frameworks can reach their performance bottleneck with minimal memory, we set their memory to the memory required for 99% HH measurement accuracy under the ideal periodic algorithm for fairness. We implemented other frameworks under default parameters and compared their report overhead with HF, as shown in Table 3. The result shows that in the MAWI and IMC datasets, the overhead of HF is significantly lower than that of periodic frameworks with a 1 ms window and even slightly lower than some frameworks with a 50 ms window. HF's overhead is also lower than OC and IPG under default settings, only exceeding that of SF, which only sends one message for the complete lifecycle of flows and cannot describe fine-grained flow behaviors. However, in the RDMA dataset, HF's overhead increases substantially, remaining lower than periodic frameworks with a 1 ms window but slightly higher than the overhead of IPG and SF. Like OC, HF's mea-

Authorized licensed use limited to: Tsinghua University. Downloaded on September 28,2025 at 13:54:14 UTC from IEEE Xplore. Restrictions apply.

surement granularity increases with flow rates, leading to more reports. Nevertheless, due to HF's compression mechanism and the limited rate variations in RDMA flows, its overhead increase is much smaller than that of OC.

VI. Conclusion

This study introduces HeavyFinder, a system intended for efficient and fine-grained heavy hitters detection in high-performance networks. Compared with existing frameworks, HeavyFinder can provide fine-grained information about flows that have a significant influence on the network to the control plane with limited overhead. To achieve this target, HeavyFinder defines a flow-rate-sensitive HH and monitors this special flow event at the microsecond level. Furthermore, we designed a rate-based elephant flow filtering technique and a HH message aggregation approach to collect the flow's impact information more comprehensively and efficiently. Experiments have validated the benefits of the above design.

VII. Acknowledgement

We thank all the reviewers for their valuable comments and advice. This work is supported by the National Natural Science Foundation of China (No. 62132004, 62221003). Yangyang Wang, Mingwei Xu are the corresponding authors.

References

[1] Mohammad Al-Fares, Sivasankar Radhakrishnan, Barath Raghavan, Nelson Huang, Amin Vahdat, et al. Hedera: dynamic flow scheduling for data center networks. In *Nsdi*, volume 10, pages 89–92. San Jose, USA, 2010.

[2] Yiran Lei, Yu Zhou, Yunsenxiao Lin, Mingwei Xu, and Yangyang Wang. Dove: Diagnosis-driven slo violation detection. In *2021 IEEE 29th International Conference on Network Protocols (ICNP)*, pages 1–11. IEEE, 2021.

[3] Kaicheng Yang, Yuhan Wu, Ruijie Miao, Tong Yang, Zirui Liu, Zicang Xu, Rui Qiu, Yikai Zhao, Hanglong Lv, Zhigang Ji, et al. Chamelemon: Shifting measurement attention as network state changes. In *Proceedings of the ACM SIGCOMM 2023 Conference*, pages 881–903, 2023.

[4] Zheng Zhong, Shen Yan, Zikun Li, Decheng Tan, Tong Yang, and Bin Cui. Burstsketch: Finding bursts in data streams. *in ACM SIGMOD*, 2021.

[5] Chris Misa, Ramakrishnan Durairajan, Arpit Gupta, Reza Rejaie, and Walter Willinger. Leveraging prefix structure to detect volumetric ddos attack signatures with programmable switches. In *2024 IEEE Symposium on Security and Privacy (SP)*, pages 4535–4553. IEEE, 2024.

[6] Chuanxiong Guo, Haitao Wu, Zhong Deng, Gaurav Soni, Jianxi Ye, Jitu Padhye, and Marina Lipshteyn. Rdma over commodity ethernet at scale. In *Proceedings of the 2016 ACM SIGCOMM Conference*, pages 202–215, 2016.

[7] Qiao Zhang, Vincent Liu, Hongyi Zeng, and Arvind Krishnamurthy. High-resolution measurement of data center microbursts. In *Proceedings of the 2017 Internet Measurement Conference*, pages 78–85, 2017.

[8] Hao Zheng, Chengyuan Huang, Xiangyu Han, Jiaqi Zheng, Xiaoliang Wang, Chen Tian, Wanchun Dou, and Guihai Chen. μmon: Empowering microsecond-level network monitoring with wavelets. In *Proceedings of the ACM SIGCOMM 2024 Conference*, pages 274–290, 2024.

[9] Ehab Ghabashneh, Yimeng Zhao, Cristian Lumezanu, Neil Spring, Srikanth Sundaresan, and Sanjay Rao. A microscopic view of bursts, buffer contention, and loss in data centers. In *Proceedings of the 22nd ACM Internet Measurement Conference*, pages 567–580, 2022.

[10] Prateesh Goyal, Preey Shah, Naveen Kr Sharma, Mohammad Alizadeh, and Thomas E Anderson. Backpressure flow control. In *Proceedings of the 2019 Workshop on Buffer Sizing*, pages 1–3, 2019.

[11] Shicheng Wang, Menghao Zhang, Yuying Du, Ziteng Chen, Zhiliang Wang, Mingwei Xu, Renjie Xie, and Jiahai Yang. Lordma: A new low-rate dos attack in rdma networks. *in NDSS*, 2024.

[12] Kaihui Gao, Dan Li, and Shuai Wang. Bandwidth-efficient microburst measurement in large-scale datacenter networks. In *Proceedings of the 6th Asia-Pacific Workshop on Networking*, pages 14–20, 2022.

[13] Xiaoqi Chen, Shir Landau Feibish, Yaron Koral, Jennifer Rexford, Ori Rottenstreich, Steven A Monetti, and Tzuu-Yi Wang. Fine-grained queue measurement in the data plane. In *Proceedings of the 15th International Conference on Emerging Networking Experiments And Technologies*, pages 15–29, 2019.

[14] Yikai Zhao, Wenchen Han, Zheng Zhong, Yinda Zhang, Tong Yang, and Bin Cui. Double-anonymous sketch: Achieving top-k-fairness for finding global top-k frequent items. *Proceedings of the ACM on Management of Data*, 1(1):1–26, 2023.

[15] Ruijie Miao, Yiyao Zhang, Guanyu Qu, Kaicheng Yang, Tong Yang, and Bin Cui. Hyper-uss: Answering subset query over multi-attribute data stream. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, pages 1698–1709, 2023.

[16] Yinda Zhang, Peiqing Chen, and Zaoxing Liu. Octosketch: Enabling real-time, continuous network monitoring over multiple cores.

[17] Qilong Shi, Chengjun Jia, Wenjun Li, Zaoxing Liu, Tong Yang, Jianan Ji, Gaogang Xie, Weizhe Zhang, and Minlan Yu. Bitmatcher: Bit-level counter adjustment for sketches. 2024.

[18] Yiran Lei, Liangcheng Yu, Vincent Liu, and Mingwei Xu. Printqueue: performance diagnosis via queue measurement in the data plane. In *Proceedings of the ACM SIGCOMM 2022 Conference*, pages 516–529, 2022.

[19] Nick Littlestone and Manfred K Warmuth. The weighted majority algorithm. *Information and computation*, 108(2):212–261, 1994.

[20] Mea Wang, Baochun Li, and Zongpeng Li. sflow: Towards resource-efficient and agile service federation in service overlay networks. In *24th International Conference on Distributed Computing Systems, 2004. Proceedings.*, pages 628–635. IEEE, 2004.

[21] Nikhil Handigol, Brandon Heller, Vimalkumar Jeyakumar, David Mazières, and Nick McKeown. I know what your packet did last hop: Using packet histories to troubleshoot networks. In *11th USENIX Symposium on Networked Systems Design and Implementation (NSDI 14)*, pages 71–85, 2014.

[22] Raj Joshi, Ting Qu, Mun Choon Chan, Ben Leong, and Boon Thau Loo. Burstradar: Practical real-time microburst monitoring for datacenter networks. In *Proceedings of the 9th Asia-Pacific Workshop on Systems*, pages 1–8, 2018.

[23] Bohan Zhao, Xiang Li, Boyu Tian, Zhiyu Mei, and Wenfei Wu. Dhs: Adaptive memory layout organization of sketch slots for fast and accurate data stream processing. *in ACM SIGKDD*, 2021.

[24] Suneet Kumar Singh, Christian Esteve Rothenberg, Marcelo Caggiani Luizelli, Gianni Antichi, Pedro Henrique Gomes, and Gergely Pongrácz. Hh-ipg: Leveraging inter-packet gap metrics in p4 hardware for heavy hitter detection. *IEEE Transactions on Network and Service Management*, 2022.

[25] Zongyi Zhao, Xingang Shi, Arpit Gupta, Qing Li, Zhiliang Wang, Bin Xiong, and Xia Yin. Continuous flow measurement with superflow. In *2021 IEEE/ACM 29th International Symposium on Quality of Service (IWQOS)*, pages 1–6. IEEE, 2021.

[26] Yinda Zhang, Peiqing Chen, and Zaoxing Liu. {OctoSketch}: Enabling {Real-Time}, continuous network monitoring over multiple cores. *in USENIX NSDI*, 2024.

[27] Jan Kučera, Diana Andreea Popescu, Han Wang, Andrew Moore, Jan Kořenek, and Gianni Antichi. Enabling event-triggered data plane monitoring. In *Proceedings of the Symposium on SDN Research*, pages 14–26, 2020.

[28] Barefoot Networks. Barefoot tofino switch. https://barefootnetworks.com/technology/.

[29] Tong Yang, Jie Jiang, Peng Liu, Qun Huang, Junzhi Gong, Yang Zhou, Rui Miao, Xiaoming Li, and Steve Uhlig. Elastic sketch: Adaptive and fast network-wide measurements. *in ACM SIGCOMM*, 2018.

[30] Mawi. Packet traces from wide backbone. https://mawi.wide.ad.jp/mawi/.

[31] Data set for imc 2010 data center measurement. https://pages.cs.wisc.edu/ tbenson/IMC10$_Data.html$.

[32] Yuliang Li, Rui Miao, Hongqiang Harry Liu, Yan Zhuang, Fei Feng, Lingbo Tang, Zheng Cao, Ming Zhang, Frank Kelly, Mohammad Alizadeh, et al. Hpcc: High precision congestion control. In *Proceedings of the ACM special interest group on data communication*, pages 44–58. 2019.

Authorized licensed use limited to: Tsinghua University. Downloaded on September 28,2025 at 13:54:14 UTC from IEEE Xplore. Restrictions apply.